

Prediction of Concrete Compressive Strength using ANN

CIL3090 Smart Infrastructure Engineering

PRESENTED TO

Dr. Ravi Prakash

PRESENTED BY

Sejal Yenage (B23CI1048)

Deepika Yampati(B23CI1047)

A decorative graphic element consisting of multiple parallel, nested zigzag lines in shades of gray, located in the bottom right corner of the slide.



Table of Contents

1 Introduction

2 Dataset Description

3 Data Preprocessing

4 Methodology

5 Results and Discussion

6 Hyperparameter Optimization

7 Conclusion

8 Future Scope

9 References

Introduction

1.1 Need for Machine Learning

Concrete is a key material in civil engineering, and its compressive strength is an important factor that affects how well structures work. It takes a lot of time and money to test strength in a lab after curing, which is the traditional way to do it. Machine learning (ML) is a useful alternative as the need for faster and more efficient decision-making grows. Artificial Neural Networks (ANNs) can model complicated nonlinear links between mix proportions and strength, which lets you make quick and accurate predictions without having to do a lot of testing.

1.2 Formulation of Hypothesis

This study is based on the hypothesis that:

An ANN model trained on mix composition and curing age can accurately predict the compressive strength of concrete.

People think that: Strength is determined by various interacting variables exhibiting nonlinear relationships.

ANN models can accurately represent these relationships. Correctly preprocessing data and tuning hyperparameters make predictions more accurate.

1.3 Database Development

The dataset utilised in this research is obtained from the UCI Machine Learning Repository.

There are 1030 samples in all.

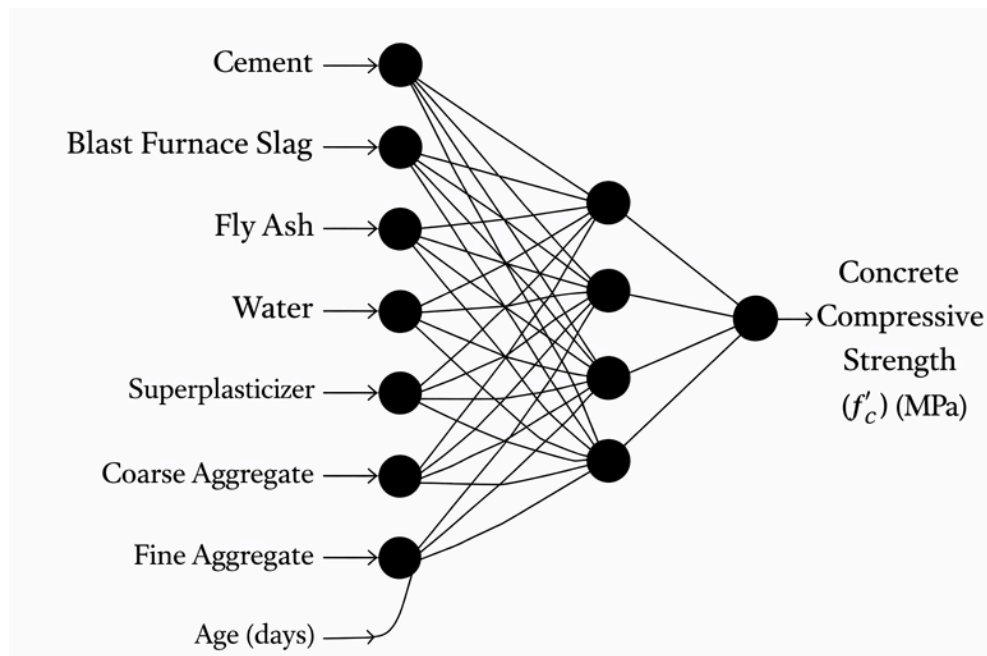
There are eight input variables: cement, blast furnace slag, fly ash, water, superplasticizer, coarse aggregate, fine aggregate, and age.

Output variable is: Compressive Strength (MPa)

The dataset includes a wide range of curing conditions and mix proportions.

There were no missing values, so all the variables are numbers, which means the data is of good quality. We sorted the data into input features and target output so that we could do more preprocessing and build a model.

Introduction



Dataset Description

The project statement says that the dataset used in this study comes from the UCI Machine Learning Repository.

Features of the Dataset:

- There are 1030 instances in total.
- Number of input features: 8
- Output variable: 1

Variables for Input

- Cement (kg/m^3) and Blast Furnace Slag (kg/m^3)
- Fly Ash (kg/m^3), Water (kg/m^3), Superplasticizer (kg/m^3), and Coarse Aggregate (kg/m^3)
- Fine Aggregate (kg/m^3) Age (days)

Variable for Output

- Strength in Compression (MPa)

The dataset includes a lot of different concrete mix compositions and curing times, which makes it good for training a model that can make general predictions.

Data Preprocessing

Data preprocessing is an important step to make sure the dataset is ready for training a neural network model. The following things were done:

3.1 Cleaning and Formatting Data

An Excel file was used to load the dataset into a pandas DataFrame. To make it easier to read, column names were made simpler by taking out units and extra characters.

We looked for missing values in the dataset. There were no missing entries, which means the data is complete.

3.2 Separating Features from Targets

The dataset was split into:

Input features (X): All columns except the last one

The target variable (y) is compressive strength.

3.3 Exploratory Data Analysis (EDA)

To understand the distribution and relationships among variables:

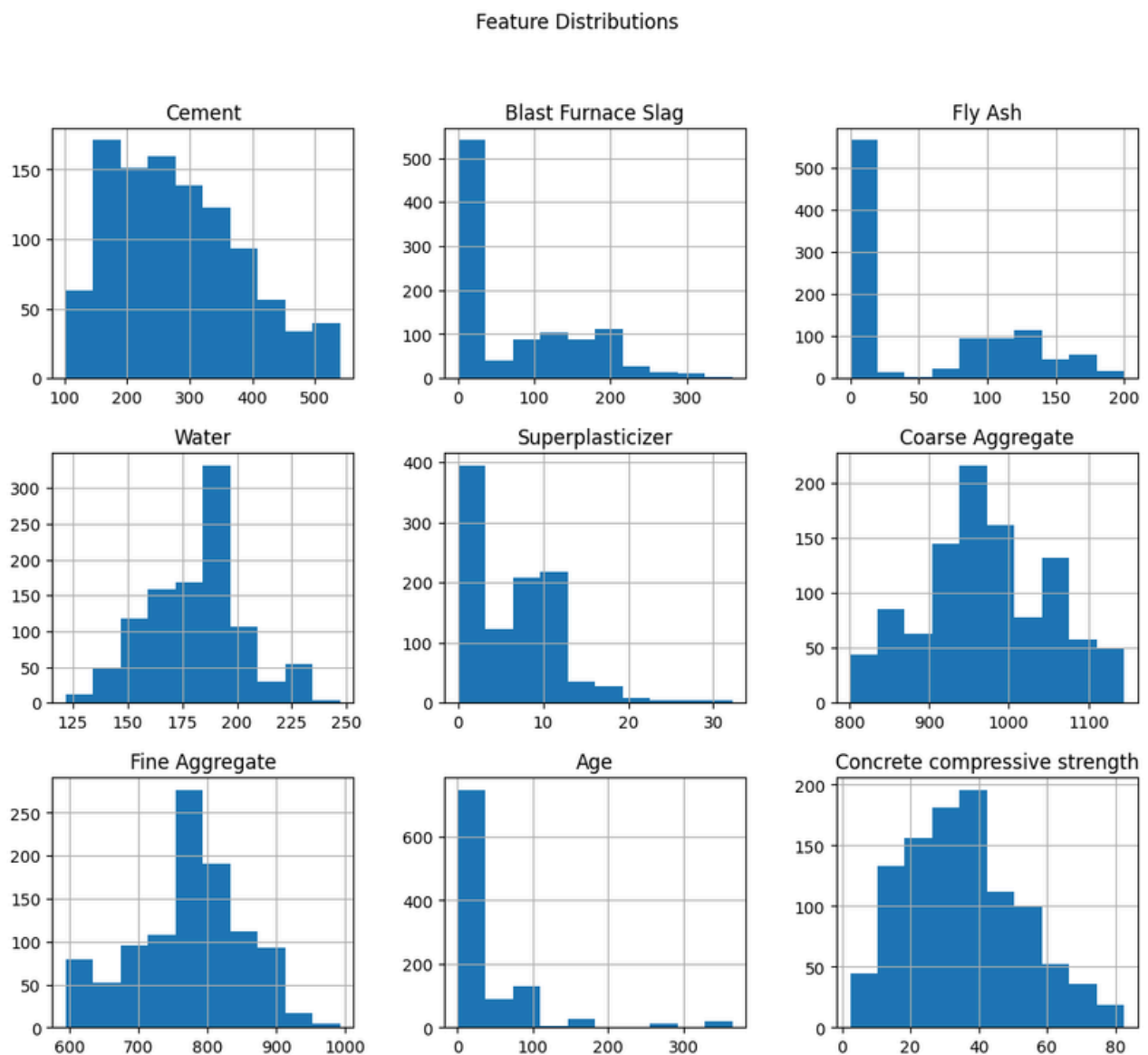
- We made histograms for all the features so we could see how they were spread out.
- We used box plots to find outliers and differences in the data.
- We made a correlation heatmap to look at how features and the target variable are related.

This analysis helped to find:

- Feature distributions
- Possible links (like cement content and strength)
- Some variables have skewness in them.

Data Preprocessing

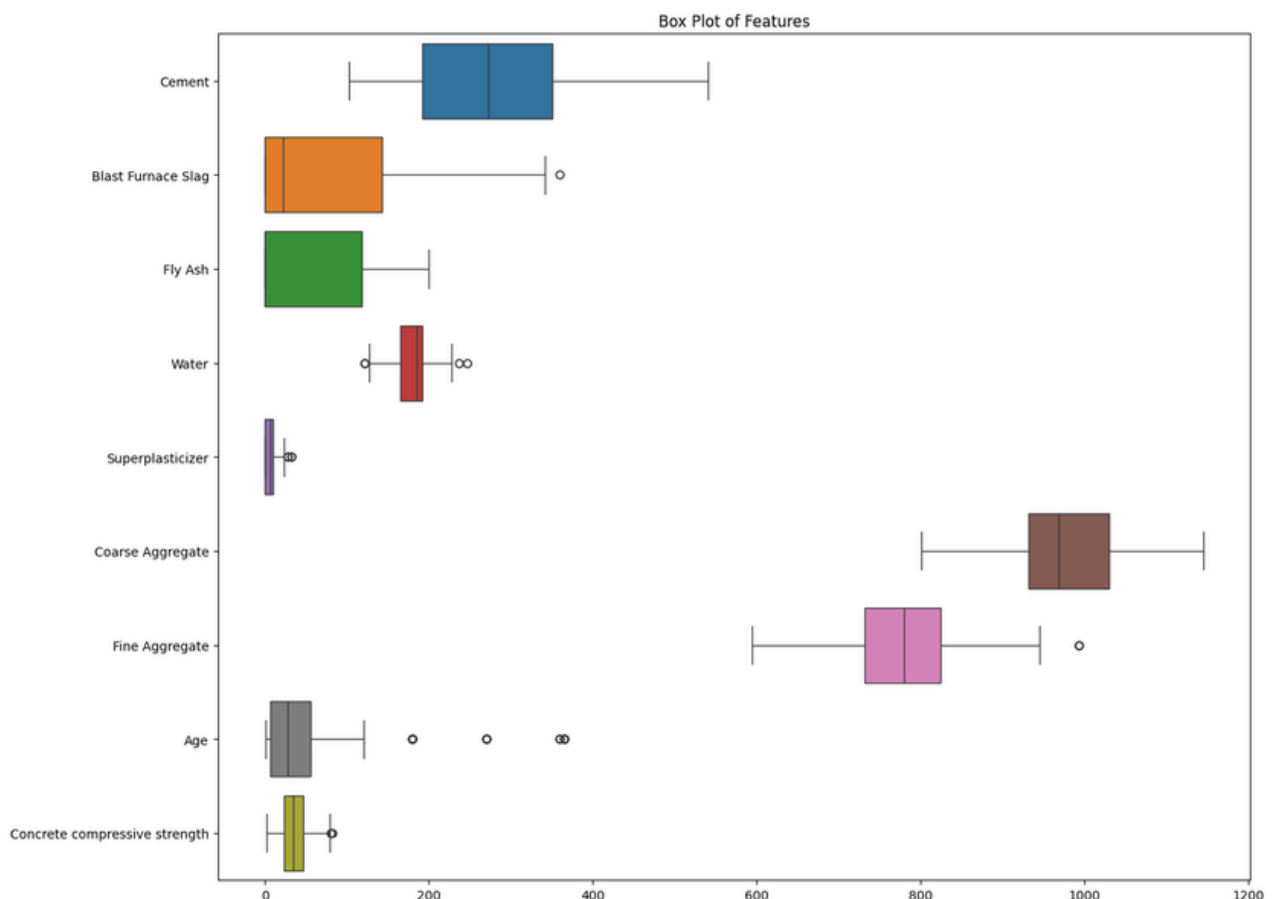
Feature Distributions:



- Most of the features are skewed, not normally distributed, so normalisation is needed.
- Slag, Fly Ash, and Superplasticizer have a lot of values that are close to zero (but not always used).
- Aggregates are mostly evenly spread out, while water doesn't change much.
- Age is mostly low (in the early curing periods).
- The compressive strength is pretty even (like a bell).

Data Preprocessing

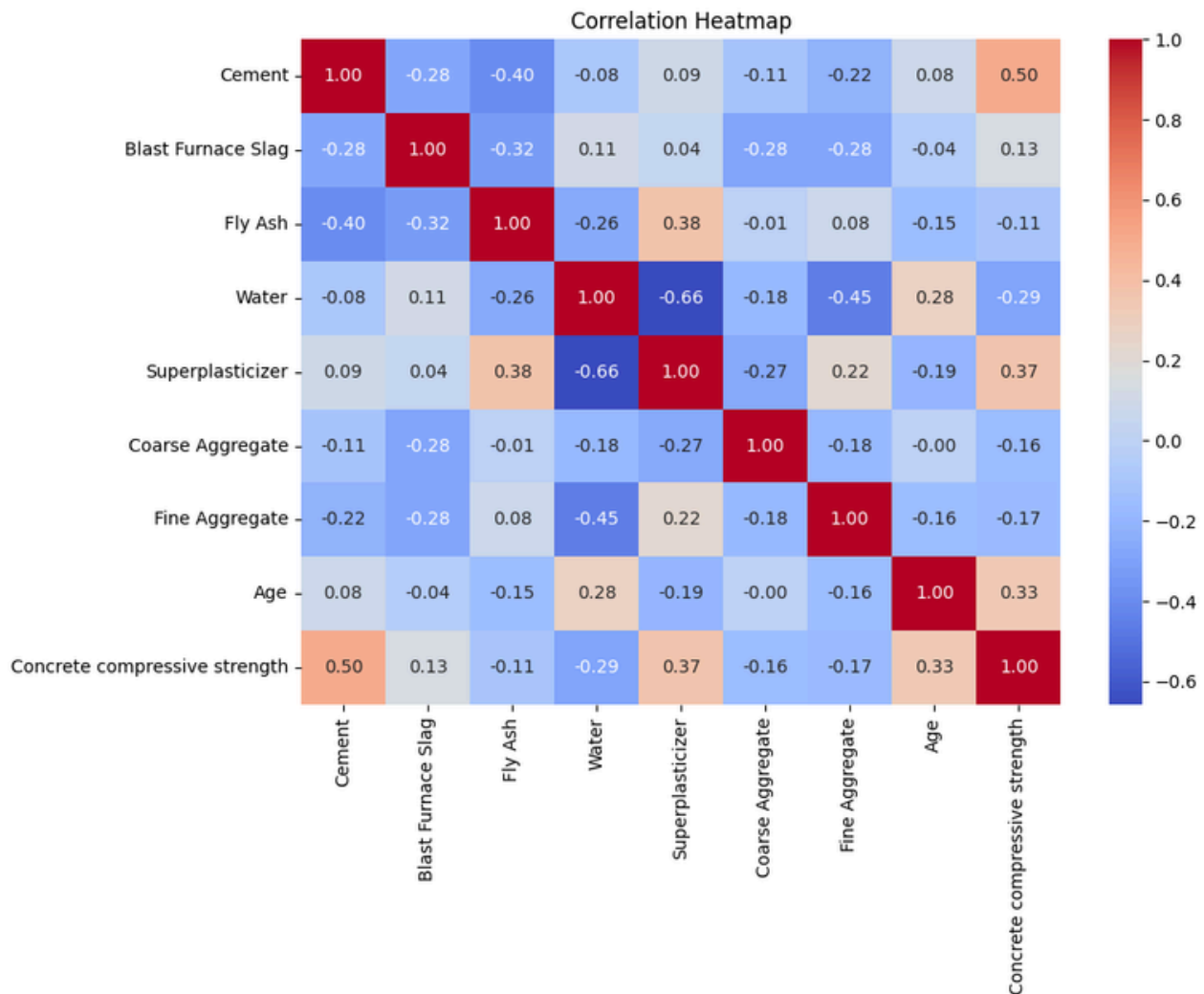
Box plots for identifying outliers:



- Age, Water, and Superplasticizer are the three features that stand out the most.
- Age has a lot of extreme values, which makes the distribution very skewed.
- Superplasticizer values are mostly low, with only a few that are very high.
- Aggregates (both coarse and fine) have a wider range but fewer outliers.
- Confirms the need for scaling and strong modelling to deal with changes.

Data Preprocessing

Correlation Heatmap:



- Cement has the strongest positive link with strength (about 0.50).
- There is a moderate positive correlation between superplasticizer and age.
- Water has a weak negative correlation with strength (-0.29).
- Most other features have weak correlations, which means that the relationships are complicated.
- Confirms that ANN needs to be able to capture nonlinear interactions

Data Preprocessing

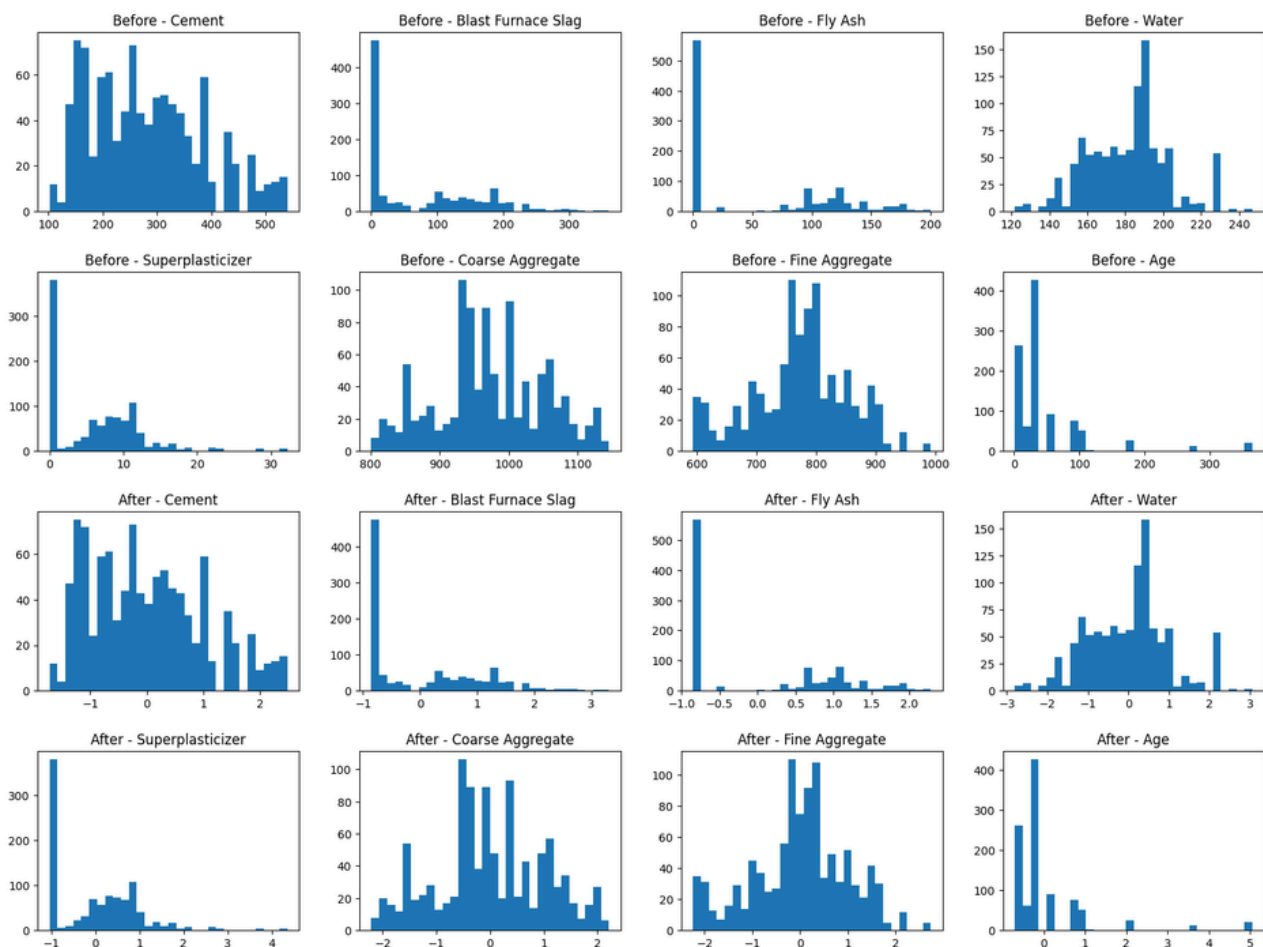
3.4 Normalisation (Scaling of Features)

Because neural networks are sensitive to the size of the input data, standardisation was used with:

$$X_{scaled} = \frac{X - \mu}{\sigma}$$

- Features that are centred around the mean and scaled by variance
- Made sure that all inputs had the same effect over training
- Faster convergence and more stable models

To check if normalisation worked, a picture of the feature distributions before and after scaling was made.



Data Preprocessing

3.5 Data Splitting

The dataset was split into three subsets:

- Training set: 70%
- Validation set: 15%
- Test set: 15%

This structure ensures:

- Training data for learning
- Validation data for tuning
- Test data for evaluation with no bias

Methodology

4.1 Choosing a Model

We chose a Feedforward Artificial Neural Network (ANN) because it can model nonlinear relationships between the input features and the output strength.

4.2 The Structure of the Baseline Model

The first baseline model has:

Input layer: The number of neurons is the same as the number of input features.

Hidden Layer 1 has 16 neurons and uses ReLU activation.

Hidden Layer 2 has 16 neurons that are activated by ReLU

One neuron (linear activation) in the output layer

Functions of Activation

ReLU (adds non-linearity) to hidden layers

Output layer: Linear (good for regression)

4.3 Putting Together the Model

The model was put together with:

Mean Squared Error (MSE) as the loss function.

Adam is the optimiser (learning rate = 0.001).

Adam was chosen because it can learn quickly and adapt.

4.4 Training Strategy

- 100 epochs
- Size of batch: 32
- Validation data used during training

Callbacks Used

- Early Stopping: checks the validation loss
- Stops training after 20 epochs if no progress
- Stops overfitting
- Terminate on NaN: stops training if invalid values are found

Methodology

4.5 Metrics for Evaluating Models

We looked at the model's performance by:

- Root Mean Squared Error (RMSE):
 - Finds out how big the prediction error is
- Mean Absolute Error (MAE):
 - Calculates the average absolute deviation
- R^2 Score (Coefficient of Determination):
 - Shows how well it fits

4.6 Visualising Performance

To look at how the model works:

- We made graphs of the training loss and validation loss curves.
- A scatter plot of predicted vs. actual strength was made.
- We could see the distribution of the residual error.

These plots help us figure out:

- Overfitting/underfitting
- How accurate are the predictions?
- Patterns of mistakes

4.7 Setting the Learning Rate

We looked into different ways to change the learning rate:

- Decay in steps
- Exponential Decay
- Cosine Decay

These change the learning rate to improve convergence make model performance better.

4.8 Tuning the Hyperparameters

To enhance the baseline model, various configurations were evaluated by altering:

- Number of neurons (16, 32, 64)
- Activation functions (ReLU, Tanh, Sigmoid)
- Optimizers (Adam, SGD, RMSprop, Adagrad)

We trained and tested each configuration on the test set and then compared the results using RMSE, MAE, and R^2 .

Model Structure

The neural network used consists of:

- Input layer: 8 features
- Hidden layers: 2 layers
- Output layer: 1 neuron (compressive strength)

Activation Functions

- ReLU: Works well and doesn't let gradients disappear
- Tanh: Smooth but takes longer to converge
- Sigmoid: Performance is limited because it is saturated.

Evaluation Metrics

- RMSE (Root Mean Square Error): shows how wrong the predictions are overall
- MAE (Mean Absolute Error): tells you how far off the average is.
- R^2 Score: shows how much variance the model captures

Data Splitting

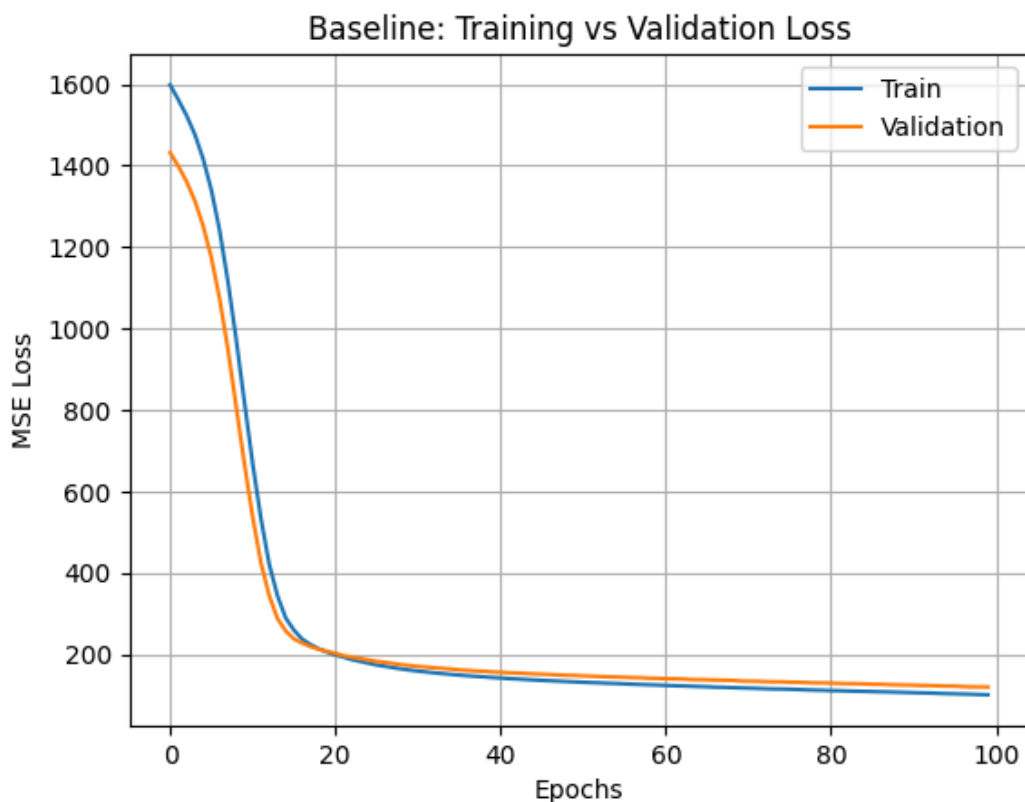
The dataset was split into:

- Training set: 70%
- Validation set: 15%
- Test set: 15%

This makes sure that the evaluation is fair and that the results can be applied to other situations.

RESULTS

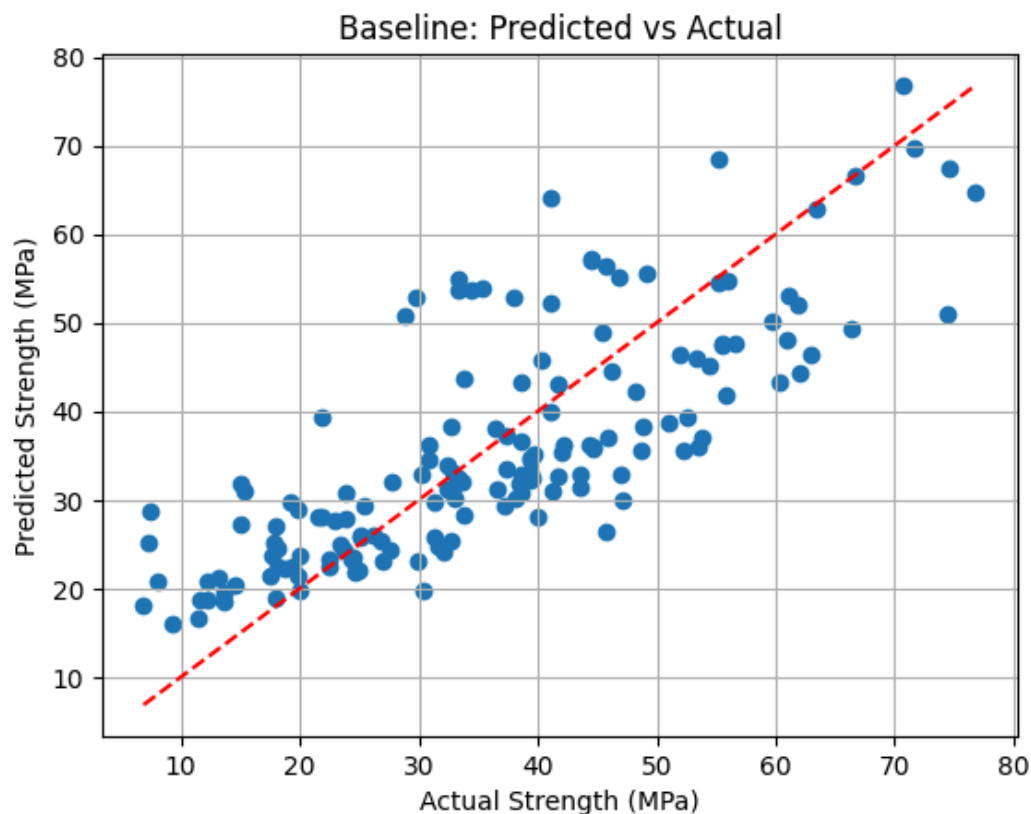
Training Behaviour



- The quick drop in loss in the first few epochs shows that learning is happening quickly.
- Stabilisation happens slowly after about 15 epochs.
- The training and validation curves are very similar.
- No noticeable overfitting was seen
- A little bit higher validation loss means a small generalisation gap.
- Smooth curves mean that the training is stable.
- The final plateau indicates learning that is almost perfect.
- A little bit of underfitting is there, which means there is room for improvement.

RESULTS

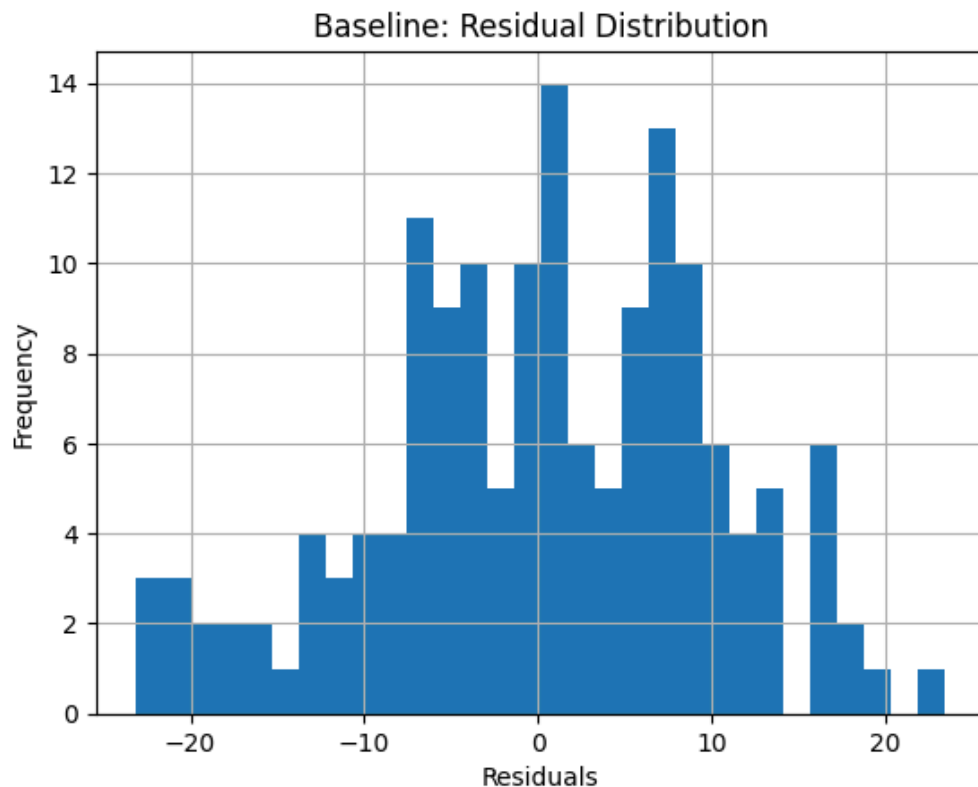
Prediction Analysis



- The blue dots show the actual and predicted values for each test sample.
- The red dashed line shows the perfect prediction (Predicted = Actual).
- Points near the red line show that the predictions are correct.
- Points away from the red line show a mistake in the prediction.
- Most of the data points follow the diagonal trend line.
- A good match in the middle range strength values
- More scatter at higher strength values
- Some underestimation and overestimation observed
- The model shows the overall trend, but not very accurately.
- Predictions get less accurate when values are very high or low.

RESULTS

Residual Analysis



- Residuals are mostly centred around zero, with a range of about -20 to +25 MPa. This means that the predictions are mostly unbiased and the error varies only slightly.
- The slight asymmetry and larger residuals show that the model is slightly biased and has trouble predicting extreme values.
- The RMSE is about 9.79 MPa and the MAE is about 7.88 MPa, which means that the model has a moderate prediction error and average deviation.
- The R^2 is about 0.62, which means that the model explains a reasonable amount of variance but not very high accuracy.
- The model gets the general trend right, but it has trouble with extreme values because it hasn't fully learned how nonlinear relationships work.

HYPERPARAMETER OPTIMIZATION

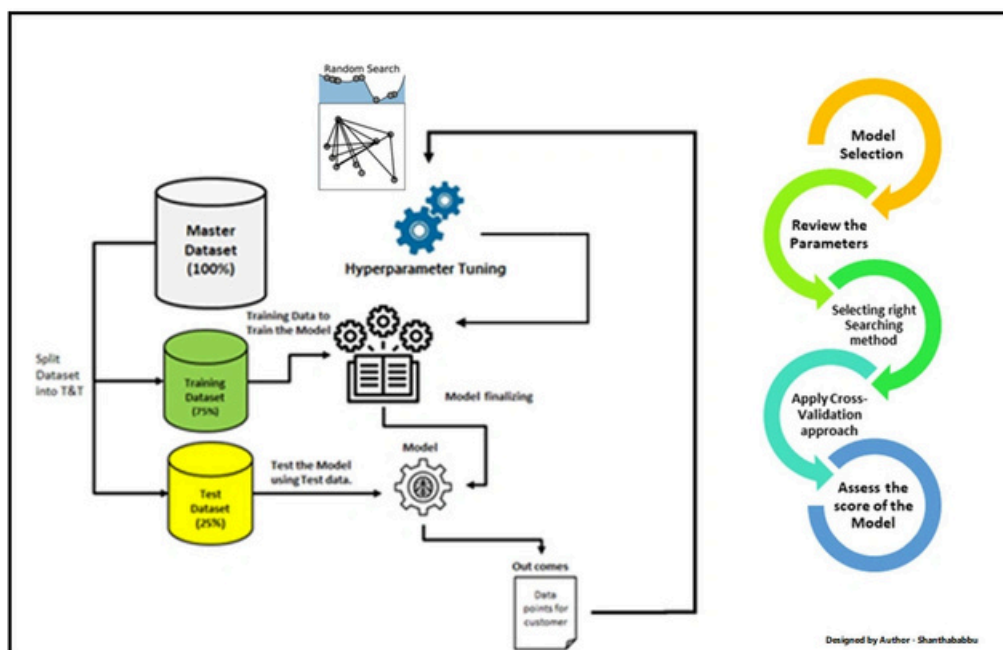
Hyperparameters Considered

The following hyperparameters were varied:

- Number of Neurons: 16, 32, 64
- Activation Functions: ReLU, Tanh, Sigmoid
- Optimizers: Adam, SGD, RMSprop, Adagrad
- Learning Rate Strategies:
 - Constant learning rate
 - Exponential decay
 - Cosine decay
- Batch Size: 32
- Epochs: 200 (with Early Stopping)

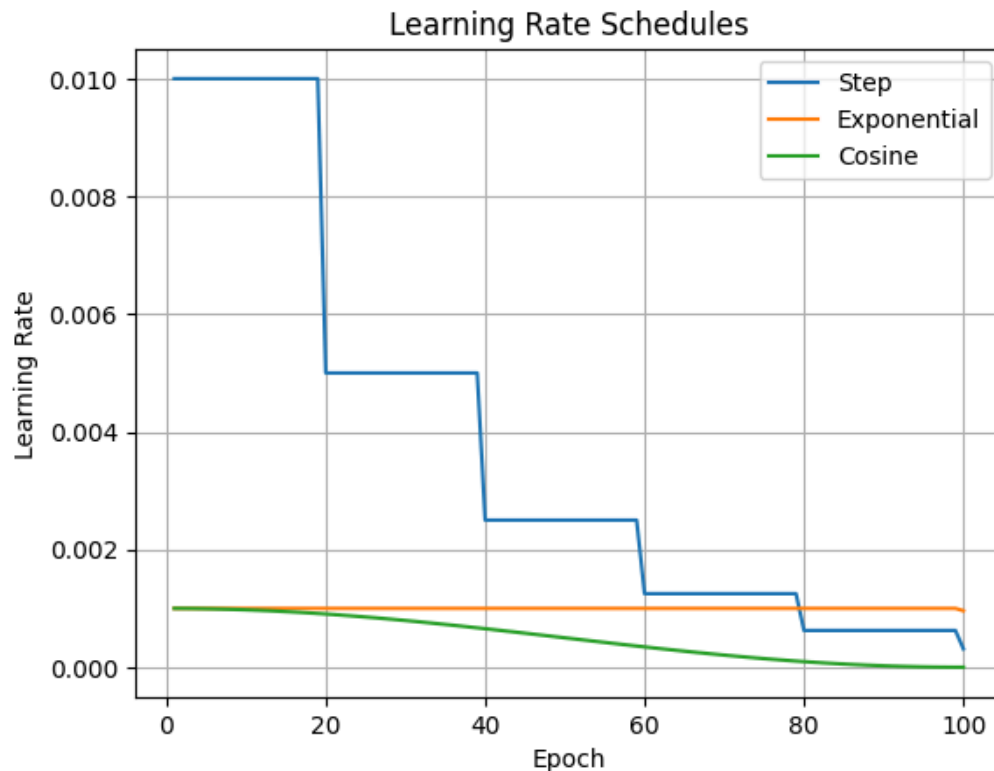
To examine the impact of learning rate variation during training, three scheduling methodologies were evaluated:

- Step Decay: cuts the learning rate at set times
- Exponential Decay: a smooth, steady drop
- Cosine Decay: changes that happen over time to improve convergence



HYPERPARAMETER OPTIMIZATION

Learning Rate Scheduling



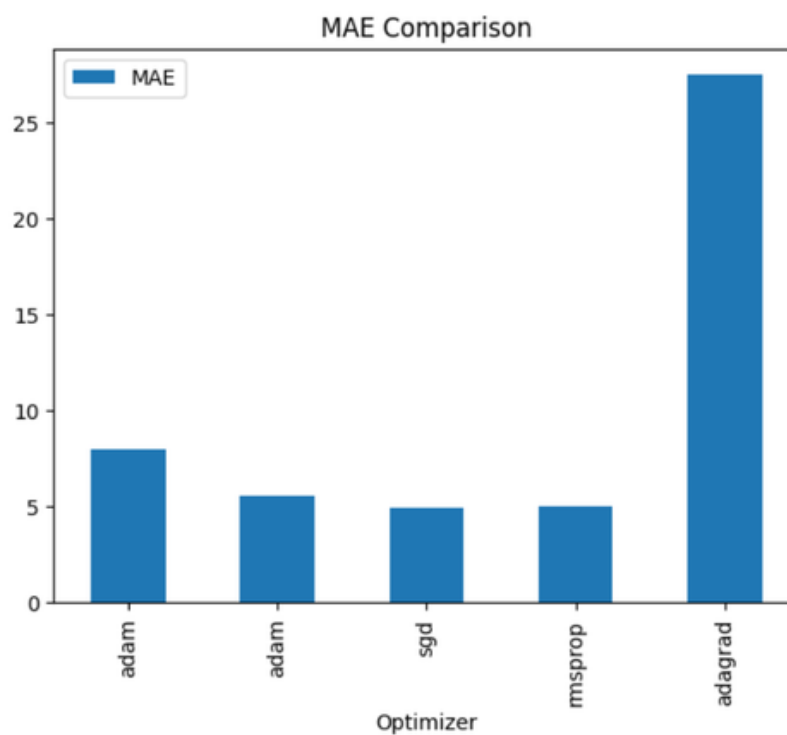
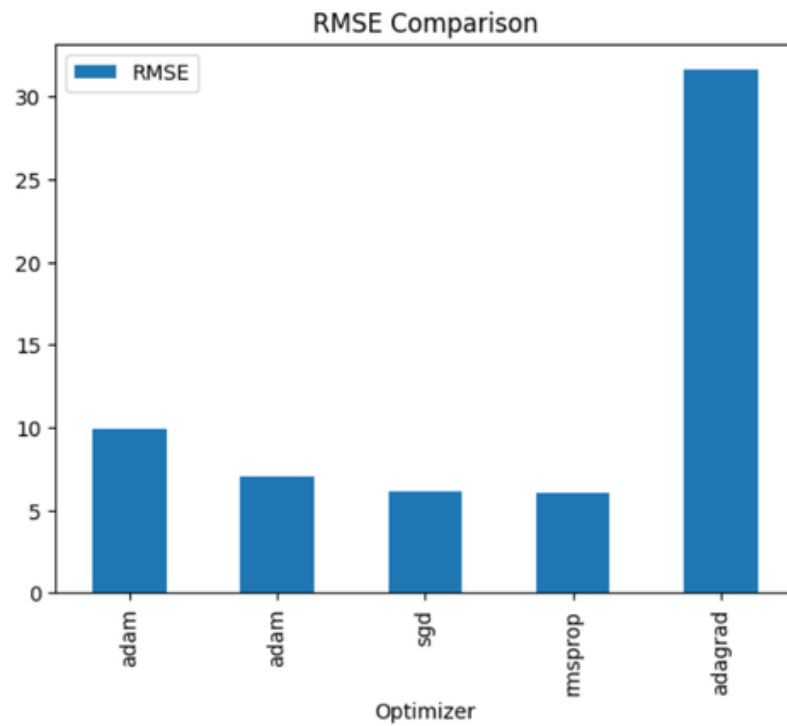
- The learning rate controls how big the weight update step is during optimisation.
- Step decay makes sudden drops at set times, which affects stability.
- Exponential decay gives a smooth and steady decrease.
- Cosine decay lets you slowly lower the value and make small adjustments later on.
- A high initial learning rate speeds up convergence, while a low rate improves final optimisation.
- Scheduling helps cut down on oscillations and makes convergence better.
- Adaptive optimisers like Adam make it less necessary to use outside scheduling.
- Learning rate schedules had little effect on how well the model worked as a whole.

Hyperparameter Optimization

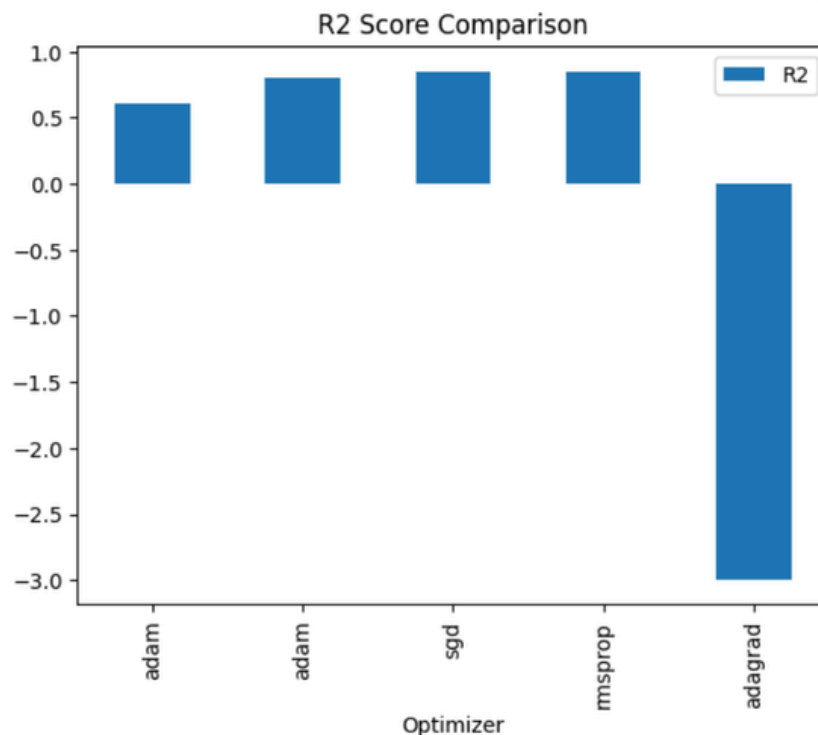
Model	Optimizer	Neurons	Activation	Performance Trend
Model 1	Adam	16	ReLU	Baseline
Model 2	Adam	32	ReLU	Improved
Model 3	SGD	32	ReLU	Poor
Model 4	RMSprop	64	Tanh	Moderate
Model 5	Adagrad	64	Sigmoid	Weak

Model	Optimizer	RMSE	MAE	R2
Model 1	Adam	9.896547	8.029826	0.608687
Model 2	Adam	7.078232	5.573392	0.799826
Model 3	SGD	6.161943	4.931233	0.848297
Model 4	RMSprop	6.084883	5.053114	0.852068
Model 5	Adagrad	31.616532	27.476094	-2.993796

Hyperparameter Optimization

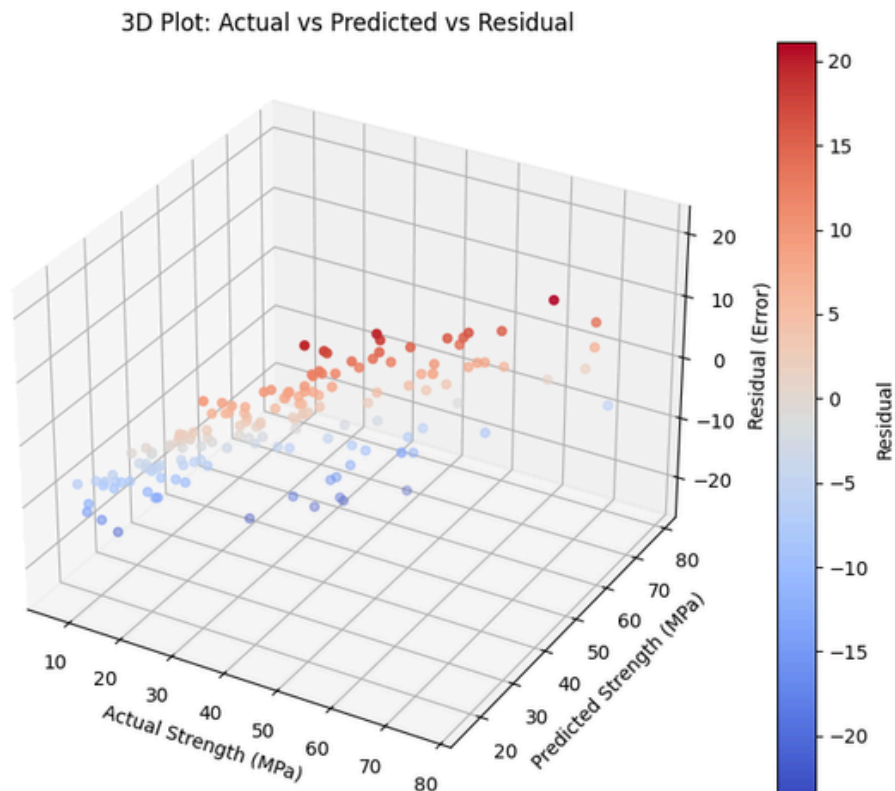


Hyperparameter Optimization



- There is a lot of difference in performance between optimisers.
- SGD and RMSprop have the best overall results.
- Adagrad does very poorly, while Adam does okay.
- SGD and RMSprop had the highest R^2 , while Adagrad had the lowest R^2 .
- SGD and RMSprop also had the lowest RMSE and MAE.
- Adagrad has very high error values.
- The choice of optimiser has a big effect on how well a model works.
- SGD and RMSprop are the best for accuracy.

Hyperparameter Optimization

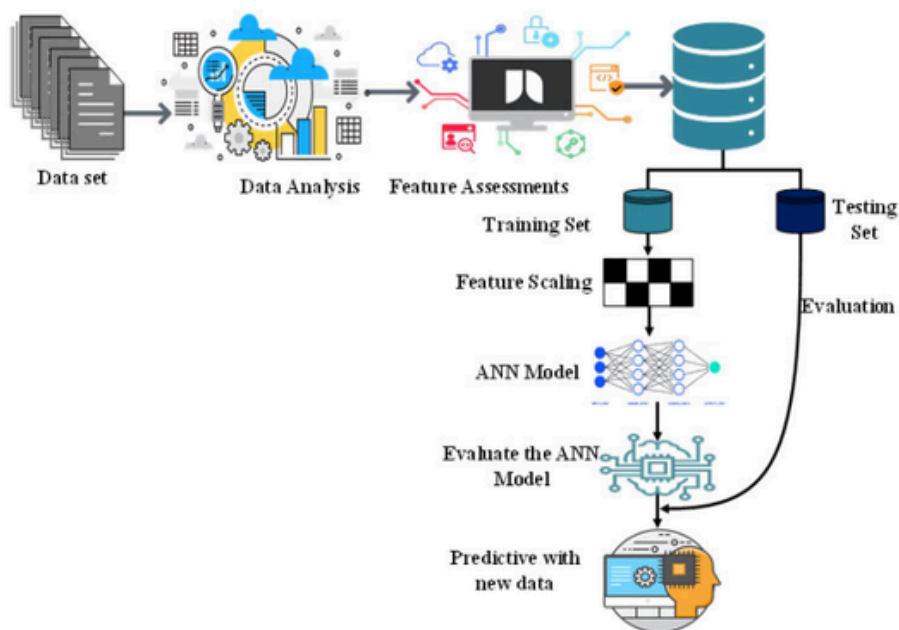


- The predicted values are very close to the actual values, which shows that the model does a good job of capturing the overall trend of compressive strength.
- Most of the residuals are close to zero, which means that the model is not very biased and is accurate for most samples.
- Some points have bigger positive and negative residuals, which means that the model has trouble accurately predicting some very high or low strength values.
- The spread of residuals gets a little bigger at higher strength values, which means that predictions are less accurate for higher compressive strengths.

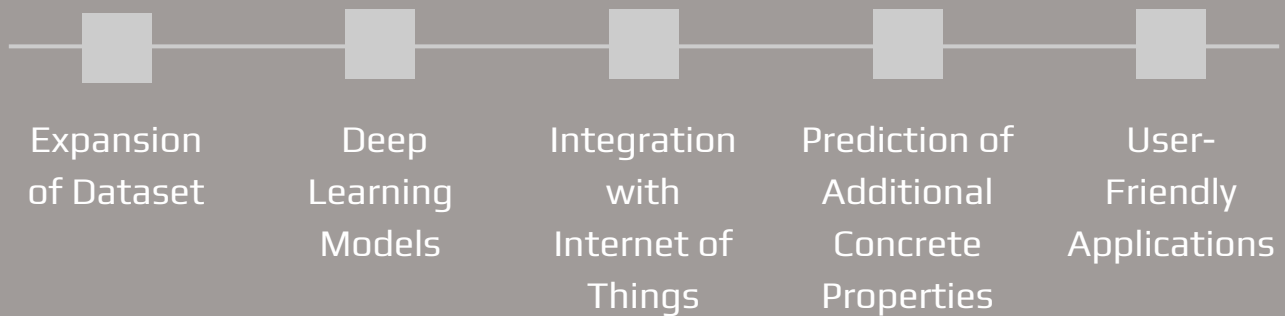
Conclusion

- This study effectively created an artificial neural network model for forecasting the compressive strength of concrete.
- Important Results
 - Baseline ANN did okay ($R^2 \approx 0.62$).
 - Adjusting hyperparameters made the model more accurate.
 - The Adam optimiser with ReLU activation worked best.
- Final Thought: Artificial Neural Networks are a good alternative to experimental testing because they save time and money while still being accurate.

Workflow



Future Scope



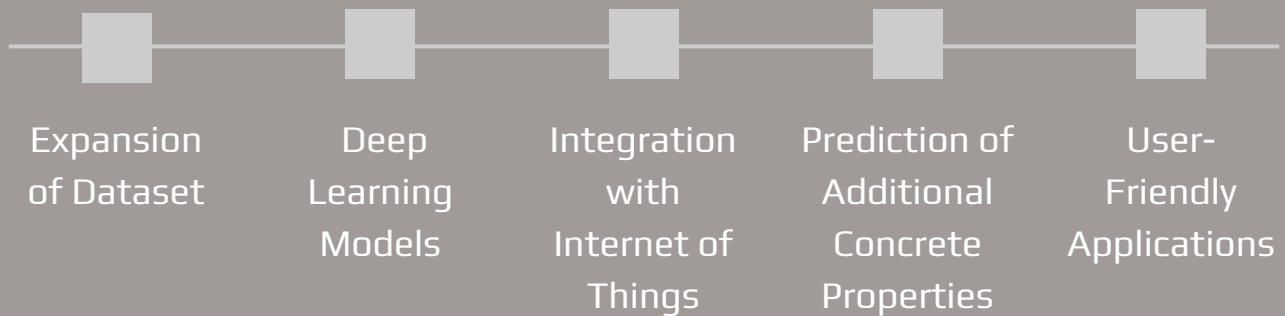
Adding to the dataset

- Use datasets that are bigger and come from more places.
- To make the model more robust, add more parameters like temperature, humidity, and curing conditions.

Advanced Methods for Deep Learning

- Look into models like Convolutional Neural Networks (CNN), Deep Neural Networks (DNN), and ensemble methods.
- Use optimisation algorithms like Genetic Algorithm (GA) and Particle Swarm Optimisation (PSO) to fine-tune hyperparameters.

Future Scope



Working with IoT

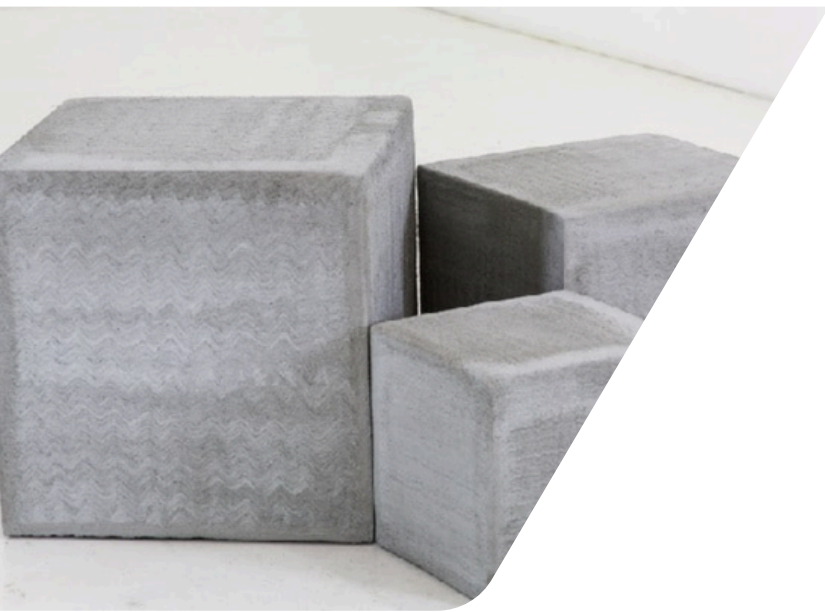
- Add built-in sensors to concrete to get data in real time.
- Allow for ongoing monitoring and on-site predictions of compressive strength.

Forecasting Extra Concrete Properties

- Add more to the model to figure out the tensile strength, modulus of elasticity, and durability.
- Create a complete system for designing smart materials.

Making apps that are easy to use

- Make web or mobile tools for engineers to quickly get strength predictions.
- Make it easier to put into practice on construction sites.



References

<https://www.mdpi.com/2076-3417/11/9/3798>

<https://doi.org/10.1007/s00521-019-04663-2>

<https://doi.org/10.1155/2015/849126>

<https://doi.org/10.1155/2016/7648467>

<https://www.proquest.com/docview/2547166562?pq-origsite=gscholar&fromopenview=true&sourcetype=Scholarly%20Journals>

Thank you

